

ЛЕКЦИЯ 12 ВИРТУАЛИЗАЦИЯ

Средства виртуализации

Виртуализация – предоставление набора ресурсов вычислительной системы или их логического объединения, абстрагированное от аппаратной реализации, и обеспечивающее при этом логическую изоляцию вычислительных процессов друг от друга, выполняемых на одном физическом ресурсе.

В качестве **объекта виртуализации** может выступать

1. Хранилище данных.
2. Приложение.
3. Вычислительная система.
4. Сеть передачи данных.

Средством виртуализации выступает программное обеспечение, которое помогает сделать вычислительные среды независимыми от физической инфраструктуры.

Наиболее ярким примером применения виртуализации является виртуализация вычислительных систем, обеспечивающая разбиение физического сервера на несколько полностью независимых виртуальных машин (ВМ) и предоставляющая возможность запуска нескольких операционных систем на одном компьютере (узле).

При этом каждый из экземпляров таких гостевых операционных систем работает со своим набором динамически выделяемых логических ресурсов (процессоры, область оперативной памяти, устройства хранения), предоставлением которых из общего пула, доступного на уровне оборудования, управляет хостовая операционная система – **гипервизор**.

Гипервизор напрямую взаимодействует с физическими ресурсами вычислителя и отвечает за то, чтобы виртуальные машины «видели» эти ресурсы как собственные. Гипервизор может в разных пропорциях распределять между ОС физические ресурсы вычислителя. Такой вид виртуализации вычислительных систем требует специальной аппаратной поддержки.

Независимая ВМ образует изолированный контейнер программного обеспечения, содержащий операционную систему и приложения. Виртуализация на уровне операционной системы обеспечивает работу нескольких экземпляров пространства пользователя в пределах одной ОС.

К достоинству виртуализации вычислительных систем следует отнести

1. Изоляцию неисправностей и нарушений системы безопасности на аппаратном уровне.
2. Сохранение уровня производительности с помощью расширенных средств управления ресурсами.
3. Сохранение состояния виртуальной машины полностью в виде файлов, дающее возможность перемещения и копирования виртуальных машин операциями с файлами.
4. Независимость ВМ от оборудования (при наличии гипервизора для данной архитектуры вычислителя) – обеспечивает простоту инициализации и миграции виртуальных машин на любой сервер.
5. Виртуализация повышает адаптивность, гибкость, масштабируемость ИТ-среды, производительность и доступность ресурсов за счет кластеризации серверов.
6. Снижает эксплуатационные расходы, минимизирует простои, существенно упрощает администрирование инфраструктуры и лицензирование программного обеспечения.
7. Обычно ВМ поддерживают технологию моментальных снимков и значительно облегчают резервное копирование и восстановление виртуальных систем.

Примерами наиболее развитых функциональных решений для виртуализации серверов являются **Microsoft Hyper-V** и **VMware vSphere**, которые требуют поддержки процессором аппаратных технологий виртуализации **Intel VT** (VT-x, Intel Virtualization Technology for x86) или **AMD-V**. Аппаратная поддержка виртуализации позволяет реализовать в многопроцессорных вычислительных системах так называемую доменную архитектуру.

Доменная архитектура основана на концепции разбиения SMP-вычислителей, предусматривающей сегментирование ресурсов вычислительной системы и создание изолированных друг от друга разделов, обеспечивающих независимую работу разных приложений.

Данная архитектура позволяет:

1. Объединить в одном вычислителе несколько полнофункциональных независимых серверных систем, каждая из которых имеет свои процессоры, память, подсистему ввода-вывода и экземпляр операционной системы.
2. Изолировать ошибки в приложениях, исполняющихся в различных разделах (они не влияют на работу прикладных систем в других разделах).

3. Исключить конкуренцию между приложениями за доступ к ресурсам вычислительной системы.
4. Обеспечить высокую масштабируемость.
5. Упростить управление совокупностью серверов.
6. Сделать прикладную среду более безопасной, изолируя аппаратные сбои и программные ошибки в разделе.
7. Снизить затраты на аппаратуру.

В современных вычислительных системах с SMP-архитектурой реализуют два типа разбиений – системное и разбиение на уровне ресурсов приложений.

Системные разделы. Системное разбиение позволяет создать на базе одного аппаратного сервера несколько разделов, каждый из которых представляет собой полнофункциональную серверную систему, работающую под управлением собственного экземпляра ОС.

Системное разбиение является необходимой функциональностью для провайдеров приложений.

Системный раздел включает все необходимые для автономной работы ОС ресурсы процессоры, память, устройства ввода-вывода.

Это позволяет одновременно

1. Выполнять эксплуатацию приложений.
2. Тестировать новые версии приложений.
3. Выполнять миграцию к новым версиям ОС и приложений.
4. Объединять в одном сервере стадии выполнения, разработки и тестирования разных приложений.

Системное разбиение является эффективным механизмом изоляции критичных корпоративных приложений, решающих задачи различных подразделений.

Кластеризация системных разделов позволяет существенно повысить надежность вычислителя, обеспечить автоматическую балансировку нагрузки с помощью динамического перераспределения клиентских соединений.

Различают статическое и динамическое разбиение.

Статическое разбиение определяет системные ресурсы, входящие в раздел (процессоры, память, шины ввода-вывода, внешние устройства), до запуска ОС, а их конфигурация не может быть изменена в ходе работы.

Динамическое разбиение позволяет создавать и удалять разделы, менять состав их ресурсов в оперативном режиме. Динамическое разбиение обеспечивает динамическое управление рабочей нагрузкой, так как конфигурацию системы можно менять в зависимости от создавшейся ситуации, передавая приложениям в период пиковых нагрузок дополнительное число процессоров.

Разделение приложений. Разделение приложений является удобным механизмом управления ресурсами вычислительной системы при предоставлении информационных услуг. Возможность закрепления за конкретным приложением или группой пользователей необходимых ресурсов упрощает достижение соглашений об уровне обслуживания (QoS). При разделении приложений наборы ресурсов для рабочих областей приложений, контролируемых одним экземпляром ОС, выделяются в "прикладные разделы". Разделение приложений дополняет системное разбиение и допустимо только в определенных аппаратных границах: на одном автономном сервере, в серверном кластере, в одном системном разделе. Следующий уровень программного разбиения обеспечивает "планирование классов" – сегментирование ресурсов прикладного раздела на несколько прикладных или пользовательских классов и явное выделение части ресурсов группе пользователей или приложений, входящих в класс. В основе разделения приложений лежит выделение некоторого числа процессорных блоков на сервере или системном разделе в процессорный набор. Каждый процессорный набор относится к одному прикладному разделу. В результате приложения и пользователи, которым выделен раздел с конкретным процессорным набором, не смогут воспользоваться ресурсами прикладного раздела с другим процессорным набором, что исключает взаимное влияние на производительность приложений, исполняемых в различных разделах. Обычно при этом процессорные наборы способны изменяться динамически за счет перемещения процессоров между наборами. Кроме выделения процессорных наборов возможно введение ограничений на объем виртуальной или физической памяти, используемой в данном прикладном разделе. Дополнительной поддержкой разделения приложений являются средства управления пропускной способностью, ограничивающие сетевой трафик приложения на уровне порта, и установление квот на дисковое пространство, используемое для прикладного раздела или класса. При этом ограничения могут быть программными или аппаратными. Для ограничений, вводимых программно, системой выдаются предупреждения о превышении квоты, а при аппаратных ограничениях запрещается запись в

файлы. Разбиение на прикладные разделы позволяет управлять приложением в случае, когда оно расходует слишком много процессорного времени или использует большой объем оперативной памяти. Разделение приложений предоставляет возможность гибкого управления ресурсами приложений для различных задач. Разбиение на уровне приложений позволяет повысить их производительность, поскольку снижает вероятность конфликтов за ресурсы. В результате появляется возможность перераспределения нагрузки системы путем переназначения ресурсов между коллективами пользователей или группами задач для балансировки рабочей нагрузки. Это делает поведение приложений более предсказуемым, что в свою очередь повышает качество информационного обслуживания и доступность приложений.